SmartHarvest - EO-driven Vineyard Monitoring

Operational Governance and Versioning

1. Purpose and scope

This document defines how the SmartHarvest MVP is operated and governed as an MLOps system, with a focus on:

- **Version control strategy** (branching, tagging, versioning of code/data/models)
- **CI/CD & automation** (regression gates tied to success criteria and artifact contracts)
- **Model lifecycle governance** (registry, reproducibility, experiment tracking)
- **Monitoring & maintenance plan** (drift proxies, incident response, operational checks)

The goal is to ensure that the MVP is reproducible, auditable, and verifiable.

---

2. Repository Access (GitHub)

This section defines how SmartHarvest is delivered and how an evaluator can access, reproduce, and inspect the MVP evidence using a GitHub-first workflow.

2.1 Repository access

The project is delivered through a GitHub repository.

- **Repository URL :**https://github.com/diiibe/MLOPS-SmartHarvest.git
- **Default branch:** main
- **Final submission tag / release:** v1.0.0-exam

2.2 Repository layout

The repository layout reflects the project's architecture:

Expected top-level structure

- **docs/**
  Documentation package, including:
  - System Specification Document (SSD),
  - Project Proposal,
  - operational notes (how to run, how to interpret outputs),
  - any required appendices (contracts, schemas, traceability).
- **backend/**
  Serving layer, including:

    - API implementation (run lifecycle endpoints),
    - run orchestration entry points.
- **frontend/**
  Web dashboard, including:
    - AOI workflow (draw/import),
    - run submission and run status views,
    - results visualization and downloads.
- **kernel/**
  Data and analytics core, including:
    - EO ingestion connectors (baseline + optional sources),
    - preprocessing and QA logic,
    - feature construction,
    - anomaly scoring kernel and conservative policy logic (when available).
- **ci/**
  Automated verification suite, structured to support:
    - unit tests (deterministic validation rules),
    - integration tests (module boundary checks),
    - regression gates (SC-aligned checks, run bundle enforcement).
- **cd/**
  Reproducibility and execution assets, including:
    - Dockerfile and docker-compose.yml,
    - scripts for demo execution,
    - environment templates (e.g., .env.example) and configuration notes.
- **runs/**
  Local run bundles created during execution. This folder is typically git-ignored and is used to store:
    - run metadata,
    - outputs and exports,
    - logs and validation reports.

Equivalent naming or comparable repository architecture is accepted.

---

3. Version Control Strategy

This section defines the version control strategy adopted to ensure that the system remains reproducible, auditable, and team-developable throughout the MVP lifecycle.

3.1 Branching model and merge discipline

Branches

Branching follows a workflow that keeps ***main*** continuously usable while allowing parallel work across data, kernel, backend, and UI.

main is the single integration branch and is treated as “always releasable”: it only contains changes that pass baseline CI and do not break contracts.

All contributions enter **main** via Pull Request with at least one peer review, using **squash merge** by default to keep history readable and traceable.

All work is done on short-lived topic branches created from **main**. We use:

- ***feature*** or ***feat/*** for new capabilities,
- ***fix/*** for repairs.

Occasionally, we use temporary specialized branches (e.g. CI/CD automation, Frontend).

Operational workflow

Development is performed using the following repeatable pattern:

1. Create a feature/* branch from main.
2. Implement the change with small, coherent commits, keeping meaningful commit messages.
3. Open a Pull Request (PR) targeting main.
4. PR checks and governance
    - CI MUST pass.
    - At least one teammate review is required.
    - Any contract-affecting change must be explicitly declared in the PR description.
5. Merge strategy
    - PRs are merged using Squash-and-merge (single commit on main) to keep history readable and to simplify traceability from release tags to change sets.

This process ensures that main remains stable, while PRs provide an auditable record of what changed and why.

Contract change rule

Because the SSD relies on stable interfaces for verification, any change that affects:

- API payload shape or endpoint behavior,
- run bundle structure (paths, filenames, required artifacts),
- schema of exports (CSV/GeoJSON),
- or policy semantics (confirmed vs candidate behavior),

shall be treated as a contract change.

Contract changes require:

1. explicit mention in the PR description,
2. a corresponding update to contract documentation,
3. and an update to CI regression tests that enforce the contract.

3.2 Tagging and release policy

Releases are tagged to allow the reproduction of the system development history, including code, contracts, and execution environment assumptions.

Tag format

v MAJOR.MINOR.PATCH label
(eg. v1.0.0-kernel)

3.3 Versioning of code and data

SmartHarvest is run-oriented: each execution is a Run, and every run must be independently auditable. Versioning is therefore defined at the run level: given a run_id, the system must be able to state exactly what execution path and inputs produced the outputs.

A) Code versioning

Each run shall record, at minimum:

- git_commit_sha (or equivalent identifier),
- container image tag or digest,
- software component versions (kernel/policy identifiers, see below).

This makes run explicitly traceable to code and supports reproducibility across environments.

B) Data versioning

EO inputs are versioned through provenance, sufficient to re-fetch and reproduce the same data request.

Each run shall record:

- AOI canonical geometry and aoi_hash,
- requested time window (start_date, end_date) and any time parameters,
- data source identifiers and provider,
- quality statistics that describe what was effectively usable (valid pixel ratio, acquisition count).

The run metadata therefore acts as a data manifest: it enables deterministic re-querying where possible, and explanation of differences where data availability changes externally.

C) Model versioning

As SmartHarvest's unsupervised-first model is the combination of:

- feature definitions,
- detector logic and version,
- and conservative post-processing policy.

Each run shall record:

- feature_set_id (feature definitions and expected shapes),
- kernel_version,
- policy_version (ensemble confirmation rules version),

- parameter set (thresholds, reliability threshold, minimum observation rules).

This ensures that anomaly outputs are interpretable and comparable across time, and that changes in logic are observable.

3.4 Contract versioning

To prevent invisible breakage during iterative development, SmartHarvest treats contracts as versioned interfaces. Contracts are tested in CI.

Contracts under version control

- **API contract**
  Versioned explicitly. A new version is introduced only when backward compatibility cannot be preserved.
- **UI-API contract (consumer expectations)**
  The UI assumes stable payload shapes from API. UI and API changes must be coordinated through versioning.
- **Run bundle contract**
  Defines mandatory artifacts, naming conventions, paths, and required exports. This is a core verifiability interface because it is inspected by CI gates.

If a contract must change, the project shall introduce a new contract version and update:

- the contract documentation,
- the CI regression checks enforcing it,
- and any dependent modules (UI/API/export tools).

Contract changes are treated as high-impact modifications that require explicit PR documentation and CI enforcement to preserve reproducibility.

---

4. CI/CD and Automation Strategy

In the SmartHarvest MVP outputs must remain conservative and auditable under incomplete Earth Observation (EO) data, therefore CI is considered the mechanism that continuously demonstrates:

- the repository remains runnable under a defined environment;
- produced artifacts respect the agreed contracts and schemas;
- project-level invariants (output completeness and conservative confirmation policies) are not violated during iterative development.

4.1 Progressive introduction

The CI pipeline is designed to be introduced progressively, coherently with the incremental delivery of system components.

Phase 1 - Data backbone:

When work primarily concerns EO ingestion, preprocessing and metadata capture, CI focuses on:

1) Reproducibility of the data backbone;
2) Validation of data contracts (AOI, target acquisition selector, minimum observation adequacy rules, schemas)

Phase 2 - End-to-end run platform:

As the ML kernel, ensemble post-processing, run orchestration, API, and UI become available, CI expands to:

1) End-to-end run execution gates;
2) Regression checks tied to Success Criteria;
3) Enforcement of the run bundle contract.

This agile staged approach avoids over-engineering before time, while ensuring each new layer is integrated under automated verification as soon as it is introduced.

4.2 Continuous Integration (CI)

Operationally, CI is executed through GitHub Actions and is configured to run automatically on:

- Pull Requests targeting main, to validate every proposed change before merge;
- Push events to main, to ensure the integrated branch remains continuously runnable and compliant with the agreed quality gates.

1. Linting and formatting

Purpose: enforce a consistent and maintainable codebase and prevent style drift, which is a common source of merge conflicts and "silent" technical debt.

Scope

- static checks for formatting and style conventions;
- basic code quality rules (e.g., unused imports, obvious anti-patterns);
- optional typing checks when applicable.

The job passes only if the codebase respects the declared standards, so that subsequent tests operate on a consistent and readable implementation.

2. Unit tests (deterministic validation rules)

Purpose: verify, with high determinism, the correctness of the rules that define the system's contracts and constraints. These rules must remain stable because they directly affect trustworthiness and verifiability of the MVP.

Scope

- **AOI contract validation:**
  Validate schema compliance and geometry constraints for AOI inputs (required fields, polygon validity, acceptable coordinate structure).

- **Target acquisition selector validity rules:**
  Validate chronological constraints and date selection constraints.

- **Minimum observation rules:**
  Verify the rule set that triggers LOW_CONFIDENCE / no-decision behavior when valid-pixel coverage / observation adequacy for the target acquisition (and minimal deterministic context if required) are insufficient.

- **Schema validators** (artifacts and exports):
  Validate that produced metadata and export payloads include required fields and respect expected structural constraints.

The job passes only if all deterministic validations behave as specified, producing pass/fail results.

3. Integration tests

Purpose: validate that separately-developed modules interoperate correctly across their interfaces, which is essential in a run-based system where pipeline components and responsibilities are partitioned.

Scope

- **Phase 1: data pipeline**
  Integration tests verify that the data acquisition and preprocessing modules can execute together under a controlled fixture and produce expected intermediate artifacts in the expected structure.

- **Phase 2: full pipeline**
  Integration tests extend to the main system boundaries, including:

  1) **Kernel - Serving Layer**: the kernel outputs match the serving layer's expected schema and semantics;
  2) **API endpoints - Run Artifacts**: API responses correctly reflect the underlying run state, metadata, and artifact availability;
  3) **UI-API contract**: the UI consumes the API payloads without schema mismatch.
  4) **Run listing contract:** list runs returns ordered runs with acquisition_date/scene_id, status, reliability summary, and minimal references to open a run.

The job passes only if module interfaces are consistent and the run lifecycle remains coherent across layers.

4.3 Regression gates (non-negotiable properties)

CI includes merge-blocking regression gates that enforce the MVP's non-negotiable properties. These gates are explicitly mapped to Success Criteria and to the run bundle contract.

Gate A - Smoke execution

Purpose: demonstrate that the repository remains runnable end-to-end without manual intervention. This gate prevents the common failure mode where components evolve independently and the integrated system becomes non-executable.

Pass condition

1. The smoke execution completes to a terminal state (SUCCESS or LOW_CONFIDENCE_DONE depending on fixture conditions).
2. The run folder is created and includes the minimal metadata needed to prove traceability (run identifier, configuration snapshot, and stage status).

Gate B - Output completeness (run bundle contract, SC2)

Purpose: ensure that every run reaching a terminal state produces a complete, inspectable run bundle, consistent with the output contract required by the MVP. This gate prevents partial outputs, missing artifacts, and "silent" failures that would undermine evaluation and reproducibility.

Pass condition

1. For every terminal run, the run bundle contains the mandatory core outputs:

    - anomaly heatmap (or equivalent raster/vector representation as defined by the project contract),
    - macro anomaly report (parcel-level summary),
    - coverage-reliability factor report.

2. The run bundle also contains the mandatory exports:

    - PDF,
    - CSV,
    - GeoJSON.

3. Artifacts are present in the expected locations and are referenced in the run metadata (paths and identifiers consistent).

Gate C - Conservative policy invariant (confirmation safety, SC3)

Purpose: enforce conservative policy by design: the system must not label anomalies as "confirmed" unless the ensemble confirmation rules are satisfied.

Pass condition.

1. No output may contain “confirmed” anomalies unless the required conditions are satisfied (multi-index coherence and persistence criteria).

Gate D - Export validity checks (schema and interoperability, SC2)

Purpose: validate that exports are structurally correct and interoperable, so that outputs can be evaluated and reused outside the platform.

Pass condition

1. GeoJSON exports are valid GeoJSON, include required fields, and load without errors in a standard GIS tool.
2. CSV exports are parseable and contain required columns and non-empty values where expected.
3. PDF exports are generated successfully (non-empty, readable) and contain the expected minimum sections.

Gate E - Acquisition anchoring and rerun determinism (scene consistency)
Purpose: enforce the acquisition-anchored run model and prevent silent changes in results when no new EO acquisition is available. This gate protects product trustworthiness by ensuring that repeated requests against the same parcel and target acquisition remain consistent and auditable.

Pass condition
Given the same AOI and the same target acquisition, two executions produce runs that:

- record the same acquisition anchoring data in run metadata (acquisition_date/scene_id),
- reference the same artifact keys in the run bundle (mandatory outputs and exports),
- preserve the conservative policy invariants (if one run is LOW_CONFIDENCE_DONE, no confirmed anomalies are present).

4.4 Continuous Delivery

Continuous Delivery for the MVP is defined as the ability to reproduce system execution in a clean environment using a single authoritative command.

The packaging strategy therefore provides a containerized execution path, notably Docker and a docker-compose template, that:

1. starts the required services (UI, API, and the run orchestrator where applicable),
2. executes a minimal on-demand demonstration workflow anchored to a target acquisition,
3. produces a complete, inspectable run bundle including mandatory outputs, exports,
4. run metadata with acquisition anchoring.

This single demo command is treated as evidence of reproducibility: it provides a consistent interface to execute the system, validate outputs, and inspect run metadata without relying on machine-specific configuration.

5. Model lifecycle governance

This section defines how SmartHarvest governs the evolution of its analytics core in a way that remains auditable, conservative, and reproducible.

5.1 Unsupervised Model Lifecycle

In the SmartHarvest, model lifecycle refers to the controlled evolution of the detection kernel and its decision logic across versions, together with the ability to reproduce and explain historical outputs. In particular, we focus on the elements that define the kernel behavior deterministically:

- **Kernel implementation version** (*kernel_version*): The code-level version of the anomaly scoring logic and any associated statistical baselines.
- **Feature set definition** *(feature_set_id*): the explicit set of indices, aggregates, and derived variables that constitute the detector input, including normalization rules and aggregation windows when applicable.
- **Policy logic version** (*policy_version*): the conservative post-processing and confirmation rules that map raw anomaly evidence into final insights.

The Model Lifecycle is treated as a controlled change-management process:

1. a kernel / feature / policy change is implemented;
2. contracts and invariants are verified through CI;
3. the change is versioned;
4. subsequent runs become reproducible and auditable through metadata records.

5.2 Experiment tracking and run registry

The fundamental unit of execution and evidence is the Run.

Every run is uniquely identified and produces a complete “run bundle” that contains both outputs and the metadata required for interpretability and reproducibility.

Each run records:

- a unique *run_id*;
- the full run configuration (AOI reference, AOI hash, target acquisition, thresholds, sources, feature set identifier);
- dataset provenance (data sources, target acquisition, and identifiers when available);
- a structured artifact registry (paths and, where applicable, checksums);
- a terminal status (SUCCESS, FAILED, LOW_CONFIDENCE_DONE);
- a structured error record when the run fails.

The run registry provides an auditable history of output production and the relative conditions.

5.3 Reproducibility policy and acceptable tolerances

Reproducibility is treated as a verification test. A run is considered reproducible if it can be re-executed using the recorded run metadata and produces outputs that remain consistent.

As mentioned above, a run shall be considered reproducible when:

1. it can be re-executed from its run metadata (AOI hash, time window, parameters and thresholds, dataset provenance references, and versions);
2. it produces the same output structure (mandatory artifacts exist, naming conventions are respected, exports are generated);

---

6. Monitoring and maintenance plan

The monitoring plan is designed for an unsupervised-first MVP, where classical label-based accuracy is not a stable primary metric. We continuously verify that the system remains runnable, conservative, auditable, and operationally meaningful under incomplete and variable EO conditions.

In SmartHarvest, monitoring has three objectives that directly support the project Success Criteria and the run-based execution model:

1. **Operational reliability of runs**
   The system must consistently reach terminal states without silent failures, and failures must be attributable to a stage and a reason.
2. **Data adequacy and data quality awareness**
   Because EO observations are frequently affected by clouds and missing acquisitions, the system must explicitly measure observation adequacy and reflect it in the run status and outputs. Monitoring must therefore distinguish *system failure* from *data limitation*.
3. **Compliance with contracts and conservative policy rules**
   Outputs must remain contract-compliant (mandatory artifacts, valid exports) and conservative (no confirmed anomalies outside policy rules). Monitoring is treated as an engineering control that detects regressions early and provides evidence of compliance.

Monitoring signals are computed from:

- **Run lifecycle events** (created, queued, running, terminal status)
- **Run metadata** (configuration, versions, provenance, quality statistics)
- **Artifact registry and validation outcomes** (presence checks, schema checks, export checks)

All signals can be derived from run bundles and logs.

6.2 Monitoring signals

The MVP implements a minimal yet meaningful monitoring baseline. Signals are grouped into three families: operational health, data quality, and compliance/completeness.

A) Operational monitoring (run health and reliability)

Operational monitoring focuses on whether the system executes predictably and whether failures are explainable.

Minimum signals:

- **Terminal-state distribution**
  Percentage and counts of terminal states: SUCCESS, FAILED, LOW_CONFIDENCE_DONE.
  Purpose: This supports SC1 (run reliability) and distinguishes data-limited outcomes from failures.

- **Run latency and runtime**
  End-to-end runtime and stage runtimes, including *p95* where feasible.
  Purpose: This supports KPI timeliness and helps detect performance regressions.

- **Failure grouping by stage**
  Failures shall be grouped by pipeline stage: INGEST, PREPROCESS, FEATURES, DETECT, RELIABILITY, EXPORT, SERVE.
  Purpose: This enables actionable debugging and prevents “unknown failure” states.

B) Data-quality monitoring

Data-quality monitoring focuses on whether EO inputs are adequate to support trustworthy outputs.

Minimum signals:

- **LOW_CONFIDENCE rate trend**
  Periodical trend of runs classified as LOW_CONFIDENCE_DONE, with attribution to main reasons (cloud coverage alto, valid_pixel_ratio basso).
  Purpose: This quantifies real-world EO limitations and supports SC4.

- **Valid pixel ratio and missingness trends**
  Trends in valid_pixel_ratio, masked coverage, acquisition counts per window, and missingness indicators.
  Purpose: These signals provide early warning for data degradation and help interpret changes in outputs.

- **Cloud-masked ratio distributions**
  Purpose: Distribution of cloud-masked ratios over runs and parcels, used to detect seasonal/cloud-driven patterns.

- **Reliability factor distribution and threshold crossings**
  Monitoring of the computed reliability factor, including the proportion of runs below the threshold and how far below they fall.
  Purpose: This provides evidence that conservative behavior is triggered consistently.

C) Compliance and completeness monitoring

Compliance monitoring ensures that the system remains verifiable even when the code evolves.

Minimum signals:

- **Artifact completeness rate**
  Percentage of terminal runs producing all mandatory artifacts (outputs and exports).
  Purpose: Any missing mandatory artifact is treated as a contract violation.

- **Export success rate and validity pass rate**
  Export generation success rates (PDF/CSV/GeoJSON) and GeoJSON validity pass rates.
  Purpose: This supports SC2 and ensures interoperability evidence.

- **Policy invariant violation count**
  Count of runs where a "confirmed" anomaly appears without satisfying the ensemble confirmation rules.
  Purpose: The expected value is zero, and any non-zero count is an error aligned with SC3.

- **Anchoring completeness rate**: percentage of terminal runs where run metadata and exports include acquisition_date and/or scene_id, and these anchoring fields are mutually consistent across artifacts (macro CSV, reliability JSON, GeoJSON).

6.3 Drift detection

In SmartHarvest, drift is not a single metric. It is treated as a warning signal indicating that either the data distribution, observation adequacy, or processing conditions may have changed in a way that could impact interpretation.

The MVP therefore monitors the following drift proxies, computed primarily on high-confidence runs to avoid confusing drift with poor EO coverage:

1. **Distribution shifts in macro summary statistics**
   Monitor shifts in parcel-level aggregates (e.g., median anomaly prevalence, summary score percentiles) across weeks.
2. **Sudden changes in LOW_CONFIDENCE frequency**
   A spike may indicate EO availability degradation, seasonal cloud patterns, or provider issues rather than model instability.
3. **Systematic changes in quality indicators**
   Monitor patterns in valid_pixel_ratio, cloud_masked_ratio, and acquisition counts by time period and parcel.
4. **Instability of run-to-run summaries not explained by adequacy**
   Detect unusually high week-to-week volatility in summaries when reliability indicators remain stable, which may suggest preprocessing instability or unintended kernel changes.

When a drift proxy triggers, we will: verify data adequacy, check preprocessing consistency and assess detector and policy version continuity.

6.4 Incident response and runbook

Incidents are grouped by the type of risk they represent for demonstration, verification, and trustworthiness.

1 - System not runnable (reproducibility failure)

Definition: the system cannot start or cannot execute the minimal demo workflow (e.g., demo command fails; API cannot start; core services not reachable).

Action:

- rollback to latest stable release tag;
- verify container/compose integrity and environment assumptions;
- re-run CI and smoke execution gate on the minimal fixture.

Evidence required:

- failing command and error log excerpt;
- the release tag used for rollback;
- the run_id and acquisition_date/scene_id (if run creation was reached) or a clear indication that it was not.

2 - Contract breakage (verifiability failure)

Definition: a run reaches a terminal state but violates the run bundle contract (missing artifacts/exports, invalid GeoJSON, schema mismatch), even if computation "worked."

Action:

- treat as a regression (merge-blocking);
- fix bundle writing/export generation/schema validation;
- add or strengthen automated test coverage for the failing condition.

Evidence required:

- run_id and acquisition_date/scene_id of an example violating run;
- artifact presence report (what is missing/invalid);
- reference to the CI gate that prevents recurrence.

3 - Data adequacy degradation (expected EO limitation)

Definition: a measurable increase in LOW_CONFIDENCE_DONE due to poor EO observations (clouds, missing acquisitions), without system failures.

Action:

- verify EO provider status and quota/availability signals;
- confirm masking thresholds and adequacy thresholds are unchanged;
- report as data limitation, not as model failure; ensure UI and exports communicate the low-confidence reason explicitly.

Evidence required:

- monitoring summary showing the spike;
- confirmation that detector/policy versions were unchanged (to exclude regressions).

For every incident class, a user-facing note shall be produced, including what happened and when; impact on user workflow and/or evaluation; mitigation steps taken and follow-up actions and links to logs and run_id(s).

6.5 Maintenance checklist

During deployment, minimum maintenance actions are required to preserve verifiability and stability:

- **Weekly monitoring review**
  Review and archive a short monitoring summary (table or log extract) reporting terminal-state distribution, runtimes, LOW_CONFIDENCE_DONE rates (with top reasons), artifact/export completeness, and anchoring completeness (acquisition_date/scene_id consistency across metadata and exports).

- **CI health verification**
  Confirm CI gates remain green on main, including smoke execution, output completeness checks, export validity checks, and the acquisition anchoring / rerun determinism gate where applicable.

- **Release reproducibility check**
  Periodically verify that the latest release remains runnable on a clean environment using the authoritative demo command and produces a complete run bundle (mandatory outputs, exports, and run metadata with acquisition anchoring) suitable for inspection.